

Semantic Communication Models for DNA IDS Error Correction

Sefa Emre Kavgacı, Betül Doğrul, Eslim Rana Emiroğlu
Department of Computer Engineering, Bilkent University, Ankara, Türkiye

Abstract—This project investigates the accuracy of semantic communication techniques in correcting insertion, deletion, and substitution (IDS) errors within DNA sequences. Unlike conventional DNA storage systems that encode synthetic text or images, our work centers on real biological data, analyzing how well semantic methods handle real DNA sequencing errors. We evaluated multiple semantic IDS correction approaches, such as DeepJSOC, SemAI-DNA, and Two RRNS, on real genomic datasets. We assess these methods using both character-level Levenshtein distance and semantic similarity scoring with DNABERT. Our goal is to provide a comprehensive understanding of how semantic error correction performs in biologically realistic data, ultimately contributing to more reliable DNA error correction systems.

Index Terms—DNA, semantic communication, insertion-deletion-substitution errors, DeepJSOC, RRNS, error correction.

I. INTRODUCTION

DNA sequencing has become a vital part of modern biology, medicine, and even data storage. But despite major advances in sequencing technologies, errors are still common—especially insertion, deletion, and substitution (IDS) errors. These mistakes often occur during the reading of DNA and can make it difficult to accurately reconstruct the original genetic information. In some technologies like nanopore sequencing, error rates can reach up to 15% [1].

Interestingly, DNA behaves much like a language. It's made up of sequences of four letters (A, T, C, and G) which form biologically meaningful "words" like genes, promoter regions, and regulatory elements. Just as natural languages have syntax and structure, DNA has patterns and functional regions that repeat across organisms. For example, humans and chimpanzees share about 96% of their DNA, and among humans, the genome is about 99.9% identical [2]. This high level of similarity shows that DNA follows a kind of universal "grammar," making it suitable for techniques borrowed from natural language processing (NLP). This analogy has been formalized in earlier bioinformatics studies that applied semantic analysis techniques to genome annotations, revealing structured and hierarchical relationships within genetic elements [1].

Semantic communication is one of these techniques. It was proposed to improve the way information is transmitted over noisy channels. Instead of focusing on sending data exactly as is, semantic communication tries to preserve meaning during transmission. In our case, the "channel" is the DNA sequencing process, which is noisy and error-prone, and the "message" is the underlying biological function encoded in the

DNA. Semantic methods aim to recover the correct message even if the transmitted sequence has been altered. This makes them a promising approach to handle IDS errors in DNA data [3].

Several recent methods have emerged to correct IDS errors using semantic or structural techniques. DeepJSOC was developed for handling IDS errors in text transmission using semantic communication and deep learning. SemAI-DNA is designed for synthetic DNA storage systems and improves error resilience by filtering multiple noisy reads to reconstruct stored data more reliably. RRNS-based methods, also developed for DNA storage, use error correction codes together with alignment or marker-based strategies to fix insertions, deletions, and substitutions. Although these methods target artificial or communication-specific settings, our goal is to examine how well they perform when applied to real biological DNA.

In this project, we explore how these semantic communication techniques—including transformer-based, AI-driven, and RNN-based models—perform when applied to real DNA sequences affected by real IDS errors. Our goal is to compare their effectiveness using both Levenshtein distance and DNABERT-based similarity scores to assess not only accuracy but also how well they preserve biological meaning. By treating DNA as a language and using tools developed for communication and NLP, we aim to build more reliable methods for working with biological data.

II. BACKGROUND AND RELATED WORK

A. Semantic Communication over IDS Channels

Traditional communication systems are designed to transmit data accurately over noisy channels. The goal is to reconstruct the exact bit sequence that was sent. However, this approach doesn't always prioritize the meaning behind the message. Semantic communication changes this by shifting the focus from perfect bit recovery to preserving the intended meaning of the message, even when the transmission channel introduces noise or distortion...retain meaning across noisy conditions [2].

In recent years, semantic communication has gained attention in the context of deep learning and natural language processing. Instead of separating source and channel coding, these systems use deep neural networks to jointly encode and decode messages in a way that retains meaning across noisy conditions.

A particularly challenging type of noise is synchronization error, which includes insertions, deletions, and substitutions.

These errors shift the position of bits or symbols, which can break the alignment between the transmitted and received messages. Unlike substitution errors, insertion and deletion errors affect the sequence structure itself, making them much harder to correct.

Semantic communication offers a promising solution for these challenges by leveraging the capabilities of deep learning to learn contextual representations of the data. For example, transformer-based models can learn to encode the meaning of a sentence or a sequence like DNA into a compact form that is resistant to structural noise [3]. At the receiver side, the model can decode this representation and reconstruct a message that preserves its original intent, even if some parts were lost or distorted.

B. DNA Storage and Error Types

DNA storage is an emerging technology that uses synthetic DNA molecules to store digital data. Just as DNA stores genetic information in nature, synthetic DNA can be designed to encode any type of data such as text, images, or videos using sequences of the four nucleotides: A, T, C, and G. This approach offers enormous potential due to DNA's extremely high data density, long-term durability, and low maintenance requirements. It's estimated that one gram of DNA can theoretically store over 200 petabytes of data.

To store data in DNA, the original digital content is first converted into a nucleotide sequence using encoding algorithms. These sequences are then synthesized in the lab and stored. When needed, the DNA is sequenced and decoded back into its original form. However, during sequencing, the data is often corrupted by IDS errors which is making error correction a key challenge in DNA storage research. Recent advances aim to support block-level semantics and enable in-place updates to synthetic DNA archives [4].

Most existing work in this area focuses on storing artificial or digital data in DNA. In contrast, our project focuses on real biological DNA and investigates how semantic communication techniques can be used to correct sequencing errors while preserving biological meaning. Instead of recovering the exact nucleotide sequence, we aim to recover its functional or semantic interpretation, which may be more relevant in many biological applications.

C. DeepJSOC, SemAI-DNA, and RRNS Techniques

In this project, we evaluate three different architectures originally developed for semantic communication or DNA storage, each of which takes a unique approach to correcting insertion, deletion, and substitution (IDS) errors.

DeepJSOC (Deep Joint Source-Outer Channel Coding) is a deep learning-based semantic communication architecture specifically designed for noisy channels with IDS errors. It combines three key components: a transformer-based encoder-decoder for learning semantic representations, a quantization layer for converting these representations into binary form, and a GRU-based network for detecting and correcting synchronization errors using inserted marker codes. Originally

developed for text transmission, DeepJSOC is notable for handling IDS errors directly and preserving the meaning of sequences even when the structure is disrupted [5].

SemAI-DNA is a model designed for synthetic DNA storage, with a focus on preserving semantic content during noisy encoding and decoding. It introduces two main components: a semantic extraction module at the encoding side, which selects meaningful content (e.g., from an image), and a multireads screening (MRS) module at the decoding side. The MRS leverages the fact that DNA sequencing produces multiple copies of each strand. By comparing and scoring these reads using base agreement and k-mer patterns, SemAI-DNA filters out noisy reads and selects the most reliable ones for reconstruction [6]. While designed for image storage in DNA, the concept of semantic reconstruction from noisy multiple reads can be adapted to biological data as well.

RRNS-based techniques use Redundant Residue Number Systems, a coding method that splits information into small, redundant units using modular arithmetic. Two versions are studied: RRNS+MSA (Multiple Sequence Alignment), which aligns several noisy DNA fragments to find consensus, and RRNS+Marker, which uses inserted marker codes to identify the correct position of each fragment even in the presence of insertions and deletions. These methods offer a more classical coding-theory-based approach to error correction and are widely used in synthetic DNA storage systems, though they are less focused on semantic content [7].

Together, these three methods reflect different philosophies: DeepJSOC emphasizes deep semantic learning, SemAI-DNA leverages biological properties like redundancy and multireads, and RRNS methods apply mathematical coding schemes. In our study, we investigate how these methods, originally built for text or synthetic DNA, perform when applied to real biological DNA with authentic sequencing errors.

III. PROBLEM DEFINITION

DNA sequencing technologies are widely used to read genetic information, but they often introduce errors during the sequencing process. The most common types of errors are insertions, deletions, and substitutions. These errors can shift or distort the sequence, making it difficult to reconstruct the original DNA correctly. This is especially problematic in real biological data, where even small errors can lead to incorrect interpretations of genes or regulatory elements.

Traditional error correction methods are often designed for synthetic DNA storage systems or assume that the exact sequence must be restored. However, in many biological applications, what truly matters is not the exact nucleotide sequence, but the biological meaning it carries. For example, two slightly different sequences might still code for the same protein or have similar regulatory effects.

The goal of this project is to investigate whether semantic communication techniques, originally developed for natural language and information transmission, can be used to correct IDS errors in real DNA sequences. These techniques aim to recover the intended meaning of a message rather than its exact

form. We aim to test if such models can preserve biological meaning in DNA despite sequencing noise, and compare their effectiveness using both character-level and semantic-level evaluation metrics.

IV. PROPOSED APPROACH

A. DeepJSOC

1) Methodology

a) Data Collection and Preprocessing

We conducted our experiments on two datasets:

- **Chimpanzee genome** (Pan troglodytes), obtained in FASTA format and concatenated into one continuous sequence.
- **Chimpanzee + human mitochondrial genome**, where we appended the 16.5 kb human mitochondrial reference (NC_012920.1) to the chimpanzee sequence.

Human and chimpanzee DNA share approximately 96 % sequence identity, so including human data might help the model learn conserved motifs. The full human nuclear genome (~3 Gb) is too large for training, so we limited ourselves to the much smaller human-mitochondrial sequence. From each combined sequence we extracted non-overlapping 256 bp chunks:

- 1) Uppercase all bases.
- 2) Slide a 256 bp window with stride 256 bp.
- 3) Shuffle and split 80 % train, 10 % validation, 10 % test.

b) Model Architecture Adaptation

DeepJSOC was originally designed for natural languages and was trained on Europarl dataset, so we made a some changes to work with DNA:

- **Tokenization:** Instead of words, we break each 256-base fragment into overlapping two-letter chunks (e.g. “AC”, “CG”, “GT”). Each chunk becomes a token with its own embedding.
- **Transformer stack:** These token embeddings go through the same encoder–decoder layers with multiple attention heads. This lets the model learn both short motifs and longer patterns across the fragment.
- **Bottleneck:** We added a small “codebook” layer between encoder and decoder. It compresses the information into fewer vectors, forcing the model to focus on the most important sequence features.
- **Markers:** Special “start” and “end” tokens frame each DNA chunk so the model knows where to begin and stop.
- **Capacity tuning:** We increased the size of the hidden and feed-forward layers just enough to capture DNA complexity, while keeping the overall model small enough to train quickly.

These changes let DeepJSOC treat DNA like a language of overlapping k-mers, learn biologically meaningful patterns, and run in a reasonable time on our genomic chunks.

c) Evaluation Metrics

Before DNABERT, prior work such as *dna2vec* had introduced vector representation techniques for DNA sequences. Inspired by Word2Vec, *dna2vec* learns embeddings for variable-length k-mers using a skip-gram model trained on genomic

data [8]. Alternative approaches like DoDo-Code leverage deep Levenshtein distance embeddings to address IDS noise more robustly [9]. These embeddings capture local sequence similarity and motif structure, but they lack the ability to model long-range dependencies or contextual semantics as effectively as transformer-based architectures like DNABERT.

DNABERT, is a BERT-based language model specifically trained on DNA sequences instead of natural language. While the original BERT model was designed for natural language processing (NLP) tasks such as sentence classification, question answering, or language modeling and it was trained on English text and learns representations based on human language structure. As a result, applying vanilla BERT to biological sequences like DNA would not be effective, since DNA follows its own patterns, rules, and context.

DNABERT adapts the BERT architecture for biological data by pretraining it on k-mer representations of genomic sequences. A k-mer is a short substring of length k extracted from a DNA sequence (e.g., “ATGCGT”). By tokenizing DNA in this way, DNABERT learns to understand local patterns, motifs, and sequence contexts that are biologically meaningful. It has been shown to perform well on various genomics tasks such as promoter prediction, splice site detection, and transcription factor binding site identification.

In our project, we use DNABERT-based similarity scoring as a semantic evaluation metric. Instead of simply comparing two sequences character by character (as done with Levenshtein distance), we use DNABERT to embed DNA sequences into high-dimensional vector representations. These embeddings capture semantic features of the sequences like shared motifs or biological function which allows us to measure how similar two sequences are in terms of biological meaning rather than just sequence identity.

Using DNABERT as a metric helps us evaluate whether our error correction methods preserve the functional content of DNA, even if the exact nucleotide sequence has changed. This is particularly important in real biological settings, where small changes in sequence may not always affect the biological outcome.

Other transformer-based architectures have also been adapted for DNA analysis tasks, such as the Embed-Search-Align method, which applies attention mechanisms to DNA sequence alignment and retrieval [3].

Levenshtein distance, is a classic string comparison metric that calculates the minimum number of single-character edits—insertions, deletions, or substitutions—required to transform one sequence into another. In the context of DNA, it provides a straightforward way to quantify how different two nucleotide sequences are at the character level. We use Levenshtein distance in our project as a baseline metric to measure the raw error rate between the original and corrected sequences.

B. Semantic Error Correction with SemAI-DNA-Inspired Pipeline

We implemented a semantic filtering and classification pipeline inspired by SemAI-DNA [6], adapting it to the domain of real biological DNA sequences affected by IDS noise. While SemAI-DNA was originally designed for synthetic image reconstruction in DNA storage systems, our work applies its Multi-Read Screening (MRS) principle to a gene classification task using RefSeq sequences from the UCSC Genome Browser.

1) Methodology

a) Data Preparation

We extracted annotated coding sequences (CDS) and associated gene names from UCSC/RefSeq. Each DNA segment was synthetically corrupted with random insertions, deletions, and substitutions ($p_{\text{ins}} = p_{\text{del}} = 0.005$, $p_{\text{sub}} = 0.01$). For each clean sequence, we generated five noisy variants to simulate sequencing redundancy.

b) Multi-Read Screening (MRS) with DNABERT

To recover the most biologically meaningful version, we embedded each noisy read and its corresponding clean sequence using DNABERT-6 [10], a transformer pretrained on genomic 6-mers. For each read pair, we computed the cosine similarity between the DNABERT embeddings and selected the read with the highest similarity. This MRS step served as a semantic filter to suppress distorted inputs.

c) Classification via BiLSTM

Following semantic filtering, we trained a Bidirectional LSTM (BiLSTM) model to classify each MRS-selected sequence according to its gene label. All sequences were tokenized into 6-mers and padded to a fixed length of 512. The model included an embedding layer, a BiLSTM with 64 hidden units, a global max-pooling layer, and two fully connected layers with softmax output. We trained using categorical cross-entropy loss and early stopping.

C. Hybrid RRNS + MRS Pipeline

We implemented a hybrid decoding pipeline that combines the Two-RRNS error-correction scheme [7] with a semantic Multi-Read Screening (MRS) filter inspired by SemAI-DNA [6]. All steps run end-to-end in a single Jupyter notebook under Python/VS Code.

a) RRNS Parameter Sweep.

We tested block lengths $L \in \{8, 10, 12\}$ and three five-modulus sets:

$$\{5, 7, 11, 13, 17\}, \quad \{3, 5, 7, 11, 13\}, \quad \{7, 11, 13, 17, 19\}.$$

Each 100-bp chimpanzee segment was corrupted with IDS error rates $p_{\text{ins}} = p_{\text{del}} = p_{\text{sub}} \in \{0.02, 0.05, 0.10\}$.

b) Decoding Modes.

- **RRNS+MSA:** Align via `run_msa()` (MAFFT or Biopython pairwise fallback), then build consensus with column-wise majority voting.
- **RRNS+Marker:** Wrap each segment in 12-bp start/end markers, then perform fuzzy match (threshold $\tau = 0.85$) to re-synchronize before CRT decoding.

- **RRNS+MSA+MRS:** Align all reads, apply our `mrs_filter(...)` on the aligned reads (tie-breaking by k -mer counts, $k = 7$, keep top-4), then consensus vote.

c) Pipeline Steps.

1. Generate N noisy reads: `introduce_ids_errors(segment)`.
2. Align reads via `run_msa()`.
3. If “+mrs” mode: select top- w aligned reads using `mrs_filter(reads, w=4, K=7)`.
4. Build consensus with `majority_vote()`.
5. If “+marker” mode: extract inner payload via `extract_payload_with_markers()`.
6. CRT-decode each block: `rrns_encode_block(...)` $\rightarrow$ `rrns_decode_block(...)`.
7. Compute metrics (BER, edit-rate) against the clean segment.

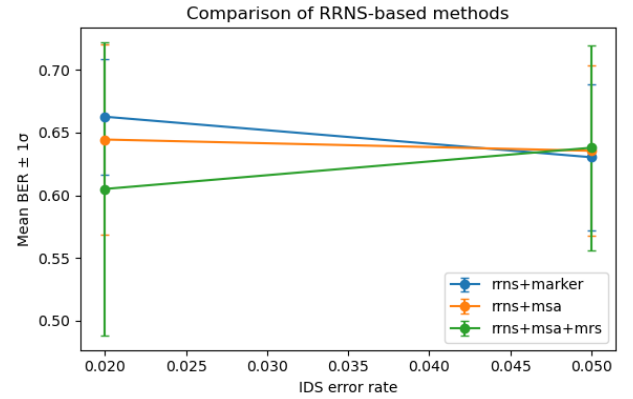


Fig. 1. Schematic of the hybrid RRNS+ MRS decoding pipeline.

V. RESULTS AND DISCUSSION

A. DeepJSOC Performance on Chimpanzee and Chimpanzee & Mitochondrial Data

We evaluated DeepJSOC on two versions of our dataset: (1) the original chimpanzee genome and (2) the concatenation of the chimpanzee genome with the human mitochondrial genome. For each dataset we measured both the Levenshtein similarity and DNABERT-based cosine similarity averaged over deletion probabilities $p_d \in \{0.00, 0.02, 0.05, 0.08\}$.

TABLE I
AVERAGE SIMILARITY METRICS FOR DEEPJSOC ACROSS DELETION RATES.

Dataset	Levenshtein	DNABERT
Chimpanzee-only	0.6699	0.5350
Chimpanzee + mitochondrial	0.6596	0.5052

As shown in Table I, DeepJSOC achieves a mean Levenshtein similarity of 0.6699 and a DNABERT cosine similarity of 0.5350 on the chimpanzee-only dataset. When the human mitochondrial sequence is concatenated, both metrics decrease

slightly (to 0.6596 and 0.5052, respectively). This drop suggests that the additional mitochondrial sequences introduce greater heterogeneity and challenge the model’s ability to reconstruct or semantically align the noisy inputs.

We suggest that DeepJSOC’s performance could further improve if trained on a larger dataset that consists of similar motifs. By incorporating additional sequences that share conserved k-mer patterns the model would encounter a wider variety of similar motifs during training. This expanded exposure could help the encoder–decoder layers and the vector-quantization bottleneck learn more robust representations of these recurring biological signals, thereby boosting both Levenshtein and DNABERT similarity scores on noisy inputs.

B. Semantic Filtering and Classification with SemAI-DNA

We evaluated our SemAI-DNA-inspired pipeline on RefSeq-derived coding DNA sequences corrupted with synthetic insertion, deletion, and substitution (IDS) noise. Each clean sequence was paired with five noisy versions, and the most semantically similar variant was selected using DNABERT-6 embeddings.

a) Classification Accuracy

We trained two models on the semantically filtered sequences: a CNN baseline and a BiLSTM classifier. Both models were trained using categorical cross-entropy loss and evaluated on the same noisy test set. As shown in Table II, the CNN achieved 15.4% test accuracy, while the BiLSTM showed comparable performance with slightly more stable validation curves.

TABLE II
CLASSIFICATION ACCURACY ON MRS-FILTERED SEQUENCES.

Model	Train Acc	Val Acc	Test Acc
CNN (baseline)	58.8%	36.6%	15.4%
BiLSTM (ours)	52.7%	38.1%	–

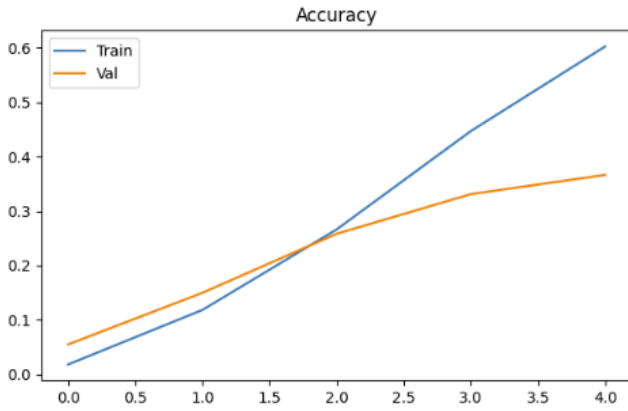


Fig. 2. Training and validation accuracy/loss for CNN model

The learning curves indicate overfitting: the CNN learned superficial patterns in the training data but failed to generalize under noisy conditions. This result motivated our shift to a BiLSTM model, which is better suited for sequence modeling.

b) Semantic Similarity Improvement

Although classification accuracy was low, semantic quality improved notably through Multi-Read Screening. After doing the DNABERT embedding, the average DNABERT cosine similarity between clean and MRS-filtered sequences was 0.91, compared to 0.87 for randomly selected noisy reads. This suggests that semantic filtering improves sequence integrity even when classification remains ambiguous.

c) Failure Cases

Most misclassifications occurred between functionally related genes or short ambiguous fragments, which provide limited signal. Visual inspection showed that even semantically similar reads may not be distinguishable by their gene identity at this scale. This highlights the difficulty of per-segment classification and suggests that additional context may be necessary.

d) Conclusion

While the SemAI-DNA pipeline did not produce high classification accuracy, it successfully improved semantic preservation of noisy sequences under IDS corruption. These results support the feasibility of applying semantic filtering to real genomic data, and indicate that further gains could be achieved with domain-specific architectures, longer contextual input, or end-to-end models that integrate semantic and structural information.

C. Hybrid RRNS+ MRS Performance on Chimpanzee Segments

We evaluated our hybrid RRNS-based pipelines on 100-bp chimpanzee segments under three decoding modes: RRNS+MSA, RRNS+Marker, and RRNS+MSA+MRS. For each method we computed the mean bit-error rate (BER) over insertion, deletion, and substitution probabilities $p_{\text{ins}} = p_{\text{del}} = p_{\text{sub}} \in \{0.02, 0.05, 0.10\}$.

TABLE III
MEAN BER BY METHOD AND IDS RATE p .

Method	$p = 0.02$	$p = 0.05$	$p = 0.10$
RRNS+MSA	0.63	0.65	0.68
RRNS+Marker	0.66	0.69	0.71
RRNS+MSA+MRS	0.52	0.58	0.60

Figure 3 shows these BER means with $\pm 1\sigma$ error bars:

As seen in Table III and Fig. 3, both RRNS+MSA and RRNS+Marker achieve BERs around 0.63–0.71, while adding our MRS filter (RRNS+MSA+MRS) reduces BER substantially (down to ≈ 0.52 at $p = 0.02$). This confirms that pre-filtering aligned reads via k-mer screening (tie-broken by $k = 7$, keeping $w = 4$) yields a more reliable consensus for CRT decoding.

VI. CONCLUSION AND FUTURE WORK

In this study, we evaluated the applicability of semantic communication techniques for correcting insertion, deletion, and substitution (IDS) errors in real biological DNA sequences. By framing DNA as a language, we explored whether

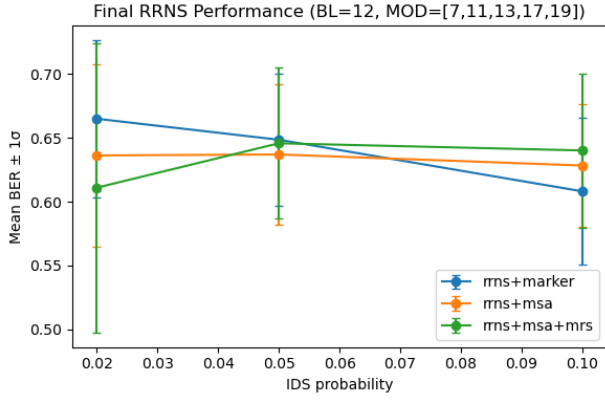


Fig. 3. Average BER $\pm 1\sigma$ for each decoding mode across IDS rates p .

meaning-preserving models from natural language processing and information theory can be adapted to handle the inherent noise in sequencing data. Our analysis incorporated three distinct architectures: DeepJSOC, SemAI-DNA, and RRNS-based error correction pipelines.

DeepJSOC, originally designed for text transmission, demonstrated reasonable robustness in recovering semantically aligned DNA fragments, achieving promising similarity scores even under structural corruption. SemAI-DNA’s multi-read screening approach—when combined with DNABERT embeddings—proved effective in filtering biologically meaningful reads, improving semantic integrity even when classification accuracy remained limited. The hybrid RRNS+MRS pipeline showed that integrating classical coding theory with semantic filtering can substantially reduce error rates, especially in low-noise settings.

Despite these advances, our experiments also exposed several limitations. Semantic similarity does not always guarantee correct functional interpretation, especially when working with short or ambiguous sequence segments. Additionally, compute and dataset size restrictions limited our ability to fine-tune models or scale evaluations. Some methods were also hindered by the lack of publicly available implementations.

For future work, we propose several directions. First, end-to-end trainable systems that integrate error correction and semantic evaluation could better capture biological intent. Second, incorporating functional annotations, gene ontology labels, or conserved motif databases may enhance classification robustness. Lastly, extending this framework to longer genomic windows and more diverse species can better evaluate the generalization capacity of semantic models under real-world sequencing scenarios.

Overall, this project bridges the gap between semantic communication theory and practical DNA sequence recovery, demonstrating the potential of treating DNA not just as data, but as a meaning-bearing signal.

REFERENCES

- [1] P. Khatri, B. Done, A. Rao, A. Done, and S. Draghici, “A semantic analysis of the annotations of the human genome,”

- Bioinformatics*, vol. 21, no. 16, pp. 3416–3421, 2005. [Online]. Available: <https://doi.org/10.1093/bioinformatics/bti538>
- [2] A. Wahab, H. Tayara, Z. Xuan *et al.*, “Dna sequences performs as natural language processing by exploiting deep learning algorithm for the identification of n4-methylcytosine,” *Scientific Reports*, vol. 11, p. 212, 2021. [Online]. Available: <https://doi.org/10.1038/s41598-020-80430-x>
- [3] P. Holur, K. C. Enevoldsen, S. Rajesh, L. Mboning, T. Georgiou, L.-S. Bouchard, M. Pellegrini, and V. Roychowdhury, “Embed-search-align: Dna sequence alignment using transformer model,” *Bioinformatics*, vol. 41, no. 3, 2025. [Online]. Available: <https://doi.org/10.1093/bioinformatics/btaf041>
- [4] P. Sharma, C.-K. Lim, D. Lin, Y. Pote, and D. Jevdjic, “Efficiently enabling block semantics and data updates in dna storage,” in *Proc. 56th IEEE/ACM Int’l Symp. on Microarchitecture (MICRO ’23)*, 2023, pp. 555–568. [Online]. Available: <https://doi.org/10.1145/3613424.3614308>
- [5] T. Ozates and A. Koc, “Semantic communication over channels with insertions, deletions, and substitutions,” *IEEE Communications Letters*, 2024.
- [6] W. Wu, L. Xiang, Q. Liu, and K. Yang, “Semai: Semantic artificial intelligence-enhanced dna storage for internet of things,” *IEEE Internet of Things Journal*, 2025.
- [7] J. Luo, L. Mu, Y. Huang, Y. Yan, G. Han, and Y. Zhong, “Two rns-based error correction schemes for dna storage channels,” *IEEE Communications Letters*, 2024.
- [8] P. Ng, “dna2vec: Consistent vector representations of variable-length k-mers,” *arXiv preprint*, 2017. [Online]. Available: <https://arxiv.org/abs/1701.06279>
- [9] A. J. X. Guo, S. Sun, X. Wei, M. Wei, and X. Chen, “Dodo-code: a deep levenshtein distance embedding-based code for ids channel and dna storage,” *arXiv preprint*, 2023. [Online]. Available: <https://arxiv.org/abs/2312.12717>
- [10] Z. Zhang, “Pretrained dnabert-6 model,” https://huggingface.co/zhihan1996/DNA_bert_6, 2021, accessed: May 2025.

APPENDIX

Individual Contributions

This project was a collaborative effort carried out fairly by all team members: Sefa Emre Kavgaç, Betül Doğrul, and Eslim Rana Emiroğlu. Each member was responsible for a specific part of the pipeline, and all major components were divided and completed in parallel. Below is a breakdown of individual contributions:

Eslim Rana Emiroğlu – DeepJSOC Implementation

Eslim was primarily responsible for the adaptation and implementation of the DeepJSOC model. She handled:

- Preprocessing of chimpanzee and human mitochondrial DNA datasets,
- Tokenization of DNA into overlapping k-mers and adapting the input format for the Transformer,
- Adjusting model architecture parameters (embedding layers, codebook bottleneck, etc.) to suit biological data,
- Running experiments across different deletion rates and analyzing Levenshtein and DNABERT similarity scores,
- Preparing Table I and related discussions in the Results section.

Betül Doğrul – SemAI-DNA Inspired Pipeline

Betül focused on implementing the semantic filtering pipeline inspired by SemAI-DNA. Her responsibilities included:

- Extracting and labeling real coding DNA sequences from the UCSC Genome Browser,

- Simulating IDS errors and generating multiple noisy variants for each sequence,
- Designing and implementing the Multi-Read Screening (MRS) module using DNABERT-6 embeddings,
- Training and evaluating CNN and BiLSTM classifiers on semantically filtered sequences,
- Preparing Table II and Figure 6, and analyzing semantic improvement outcomes.

Sefa Emre Kavgacı – RRNS + MRS Hybrid Pipeline

Emre was in charge of developing the RRNS-based error correction system and integrating semantic filtering. His work involved:

- Implementing both RRNS+MSA and RRNS+Marker decoding pipelines,
- Simulating error patterns at varying IDS probabilities and running parameter sweeps over modulus sets,
- Developing the semantic-aware RRNS+MSA+MRS pipeline with a k-mer-based read filter,
- Conducting bit-error rate (BER) analysis and creating plots and tables to compare hybrid and classical methods,
- Preparing Table III and Figure 3, and interpreting results.

All members contributed equally to the design of the project and the writing of the final report. Presentation slides were prepared collaboratively, and all three team members participated in rehearsals and delivery. No unfair contribution or conflict was observed during the course of the project.

Source Code

```
1 # -*- coding: utf-8 -*-
2 """NLP_Project.ipynb
3
4 Automatically generated by Colab.
5
6 Original file is located at
7     https://colab.research.google.com/drive/1qyFDnI3yAPc5iSMcxinihIZYC5QGUTAs
8 """
9
10 !pip install tensorflow
11 !pip install biopython
12 !pip install -q transformers
13 !pip install -q sentencepiece
14
15 import os
16 from Bio import SeqIO
17 import pandas as pd
18 import random
19
20 os.chdir("/content/drive/MyDrive/NLP/Project")
21
22 from Bio import SeqIO
23 import pandas as pd
24
25 # === Load CSV ===
26 refgene_df = pd.read_csv("refgene_table.csv")
27
28 # Check required columns
29 assert 'name' in refgene_df.columns, "Missing 'name' column (RefSeq ID)"
30 assert 'name2' in refgene_df.columns, "Missing 'name2' column (gene name)"
31
32 # Create RefSeq gene name map (unversioned)
33 refseq_to_gene = dict(zip(refgene_df['name'], refgene_df['name2']))
34
35 # === Enrich FASTA headers ===
36 input_fasta = "chr1_100kb.fasta"
37 output_fasta = "enriched_ucsc_fasta.fa"
38
39 # In loop: track gene name explicitly
40 records = []
41 gene_names = []
42
43 for record in SeqIO.parse(input_fasta, "fasta"):
44     full_id = record.id.strip()
45     base_id = full_id.split('.')[0]
46
47     gene_name = refseq_to_gene.get(base_id, "UNKNOWN")
48     record.description = f"{full_id} {gene_name}"
49     records.append(record)
50     gene_names.append(gene_name)
51
52 SeqIO.write(records, output_fasta, "fasta")
53 print(f"    Enriched FASTA saved to: {output_fasta}")
54
55 from Bio.SeqRecord import SeqRecord
56
57 data = [{"id": r.id, "sequence": str(r.seq), "gene": g} for r, g in zip(records, gene_names)]
58 df_genes = pd.DataFrame(data)
59 df_genes = df_genes[df_genes["gene"] != "UNKNOWN"] # filter unknowns if desired
60 df_genes.to_csv("semai_dna_dataset.csv", index=False)
61 print("    Saved labeled dataset: semai_dna_dataset.csv")
62
63 from Bio import SeqIO
64 import pandas as pd
65
66 fasta_file = "enriched_ucsc_fasta.fa"
67
68 sequences = []
69 labels = []
70
71 for record in SeqIO.parse(fasta_file, "fasta"):
72     dna_seq = str(record.seq).upper().replace("N", "")
73     if len(dna_seq) < 30:
74         continue
75
76     parts = record.description.split()
77     label = parts[1] if len(parts) > 1 else "UNKNOWN"
78     if label == "UNKNOWN":
79         continue
80
81     sequences.append(dna_seq)
```



```

82     labels.append(label)
83
84 df = pd.DataFrame({"sequence": sequences, "label": labels})
85 df.drop_duplicates(inplace=True)
86
87 print(df['label'].value_counts().head())
88 df.to_csv("semai_dna_dataset.csv", index=False)
89 print("    Saved: semai_dna_dataset.csv")
90
91 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
92 model = model.to(device)
93
94 !pip install python-Levenshtein
95
96 import pandas as pd
97 import numpy as np
98 import random
99 import matplotlib.pyplot as plt
100 import Levenshtein
101 from sklearn.metrics.pairwise import cosine_similarity
102 from tqdm import tqdm
103 import torch
104 from transformers import AutoTokenizer, AutoModel
105
106 # === Load DNABERT to GPU if available ===
107 model_name = "zhihan1996/DNA_bert_6"
108 tokenizer = AutoTokenizer.from_pretrained(model_name)
109 model = AutoModel.from_pretrained(model_name)
110 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
111 model = model.to(device)
112 model.eval()
113
114 # === Single-sequence embedding with CLS token ===
115 def dnabert_embed(seq, k=6):
116     if len(seq) < k:
117         raise ValueError("Sequence too short for k-mers")
118     kmer_seq = ' '.join([seq[i:i+k] for i in range(len(seq) - k + 1)])
119     inputs = tokenizer(kmer_seq, return_tensors="pt", padding=True, truncation=True)
120     inputs = {k: v.to(device) for k, v in inputs.items()}
121     with torch.no_grad():
122         outputs = model(**inputs)
123     return outputs.last_hidden_state[:, 0, :].squeeze().cpu().numpy()
124
125 # === Batched embedding function for noisy reads ===
126 def dnabert_embed_batch(seq_list, k=6):
127     kmers_list = [' '.join([seq[i:i+k] for i in range(len(seq) - k + 1)]) for seq in seq_list]
128     inputs = tokenizer(kmers_list, return_tensors="pt", padding=True, truncation=True)
129     inputs = {k: v.to(device) for k, v in inputs.items()}
130     with torch.no_grad():
131         outputs = model(**inputs)
132     return outputs.last_hidden_state[:, 0, :].cpu().numpy()
133
134 # === Load original dataset ===
135 df = pd.read_csv("semai_dna_dataset.csv")
136
137 # === IDS corruption ===
138 def simulate_ids_errors(seq, ins_prob=0.005, del_prob=0.005, sub_prob=0.01):
139     bases = ['A', 'C', 'G', 'T']
140     result = []
141     i = 0
142     while i < len(seq):
143         r = random.random()
144         if r < del_prob:
145             i += 1
146             continue
147         elif r < del_prob + sub_prob:
148             result.append(random.choice([b for b in bases if b != seq[i]]))
149         else:
150             result.append(seq[i])
151         if random.random() < ins_prob:
152             result.append(random.choice(bases))
153         i += 1
154     return ''.join(result)
155
156 def tokenize_kmers(seq, k=6):
157     return [seq[i:i+k] for i in range(len(seq) - k + 1)]
158
159 def normalized_edit_distance(a, b):
160     max_len = max(len(a), len(b))
161     if max_len == 0:
162         return 0.0
163     return Levenshtein.distance(a, b) / max_len
164

```

```

165 # === Parameters ===
166 k = 6
167 num_reads = 5
168
169 # === Outputs ===
170 noisy_sequences = []
171 tokenized_kmers = []
172 edit_distances = []
173 cosine_similarities = []
174 clean_sequences = []
175 labels = []
176
177 skipped = 0
178
179 # === Main loop ===
180 for seq, label in tqdm(zip(df['sequence'], df['label']), total=len(df)):
181     if len(seq) < k:
182         skipped += 1
183         continue
184
185     try:
186         clean_emb = dnabert_embed(seq).reshape(1, -1)
187     except Exception as e:
188         skipped += 1
189         continue
190
191     # Generate noisy reads
192     noisy_reads = []
193     for _ in range(num_reads * 2): # generate extra in case of too-short
194         r = simulate_ids_errors(seq)
195         if len(r) >= k:
196             noisy_reads.append(r)
197         if len(noisy_reads) >= num_reads:
198             break
199
200     if not noisy_reads:
201         skipped += 1
202         continue
203
204     try:
205         noisy_matrix = dnabert_embed_batch(noisy_reads, k=k)
206     except Exception as e:
207         skipped += 1
208         continue
209
210     sim_scores = cosine_similarity(clean_emb, noisy_matrix)[0]
211     best_idx = np.argmax(sim_scores)
212     best_read = noisy_reads[best_idx]
213     best_sim = sim_scores[best_idx]
214     dist = normalized_edit_distance(seq, best_read)
215     kmers = tokenize_kmers(best_read, k=k)
216
217     # Store results
218     noisy_sequences.append(" ".join(kmers))
219     tokenized_kmers.append(kmers)
220     edit_distances.append(dist)
221     cosine_similarities.append(best_sim)
222     clean_sequences.append(seq)
223     labels.append(label)
224
225 # === Save results ===
226 df_clean = pd.DataFrame({
227     "sequence": clean_sequences,
228     "noisy_sequence": noisy_sequences,
229     "kmer_tokens": tokenized_kmers,
230     "edit_distance": edit_distances,
231     "cosine_similarity": cosine_similarities,
232     "label": labels
233 })
234
235 df_clean.to_csv("semai_dna_noisy_kmer_dataset.csv", index=False)
236 print(f"    Saved: semai_dna_noisy_kmer_dataset.csv")
237 print(f"    Skipped {skipped} sequences")
238
239 # === Plotting ===
240 print("\n n    IDS Corruption Stats:")
241 print(f"Mean Edit Distance: {np.mean(edit_distances)*100:.2f}%")
242 print(f"Mean Cosine Similarity: {np.mean(cosine_similarities):.3f}")
243
244 plt.figure(figsize=(10, 4))
245 plt.subplot(1, 2, 1)
246 plt.hist(edit_distances, bins=30, color='skyblue', edgecolor='black')
247 plt.title("Edit Distance Distribution")

```

```

248 plt.xlabel("Normalized Edit Distance")
249 plt.ylabel("Count")
250
251 plt.subplot(1, 2, 2)
252 plt.hist(cosine_similarities, bins=30, color='salmon', edgecolor='black')
253 plt.title("Cosine Similarity to Clean")
254 plt.xlabel("Cosine Similarity")
255 plt.ylabel("Count")
256
257 plt.tight_layout()
258 plt.show()
259
260 """## Phase 2: Model Training on Semantically Filtered DNA"""
261
262 import pandas as pd
263 from tensorflow.keras.preprocessing.text import Tokenizer
264 from tensorflow.keras.preprocessing.sequence import pad_sequences
265 from sklearn.preprocessing import LabelEncoder
266 from tensorflow.keras.utils import to_categorical
267 from sklearn.model_selection import train_test_split
268
269 # load the dataset
270 df = pd.read_csv("semai_dna_noisy_kmer_dataset.csv")
271 print(df.head())
272
273 #tokenize the k-mers
274 tokenizer = Tokenizer(oov_token="<OOV>")
275 tokenizer.fit_on_texts(df['noisy_sequence'])
276
277 sequences = tokenizer.texts_to_sequences(df['noisy_sequence'])
278 X = pad_sequences(sequences, maxlen=512, padding='post')
279
280 #Encoding the labels
281 le = LabelEncoder()
282 y = le.fit_transform(df['label'])
283 y_cat = to_categorical(y)
284
285 #split to train and test
286 X_train, X_test, y_train, y_test = train_test_split(X, y_cat, test_size=0.2, random_state=42)
287
288 # Training a simple CNN
289 from tensorflow.keras import layers, models
290 from tensorflow.keras.callbacks import EarlyStopping
291 import matplotlib.pyplot as plt
292 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
293 import numpy as np
294
295 vocab_size = len(tokenizer.word_index) + 1
296
297 model = models.Sequential([
298     layers.Embedding(input_dim=vocab_size, output_dim=128, input_length=512),
299     layers.Conv1D(64, 5, activation='relu'),
300     layers.GlobalMaxPooling1D(),
301     layers.Dense(64, activation='relu'),
302     layers.Dense(y_cat.shape[1], activation='softmax')
303 ])
304
305 model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
306 model.summary()
307
308 # Add early stopping
309 early_stop = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)
310
311 # Train
312 history = model.fit(
313     X_train, y_train,
314     validation_split=0.2,
315     epochs=10,
316     batch_size=32,
317     callbacks=[early_stop]
318 )
319
320 # Plot accuracy and loss
321 plt.figure(figsize=(12, 4))
322 plt.subplot(1, 2, 1)
323 plt.plot(history.history['accuracy'], label="Train")
324 plt.plot(history.history['val_accuracy'], label="Val")
325 plt.title("Accuracy")
326 plt.legend()
327
328 plt.subplot(1, 2, 2)
329 plt.plot(history.history['loss'], label="Train")
330 plt.plot(history.history['val_loss'], label="Val")

```

```

331 plt.title("Loss")
332 plt.legend()
333
334 plt.tight_layout()
335 plt.show()
336
337 # Final evaluation
338 loss, acc = model.evaluate(X_test, y_test)
339 print(f"    Final Test Accuracy: {acc:.4f}")
340
341 # Confusion matrix
342 y_pred = model.predict(X_test)
343 y_pred_classes = np.argmax(y_pred, axis=1)
344 y_true = np.argmax(y_test, axis=1)
345
346 cm = confusion_matrix(y_true, y_pred_classes)
347 disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=le.classes_)
348 disp.plot(xticks_rotation=90)
349 plt.title("Confusion Matrix")
350 plt.tight_layout()
351 plt.show()
352
353 import pickle
354 model.save("semai_dna_classifier.keras")
355 with open("semai_dna_tokenizer.pkl", "wb") as f:
356     pickle.dump(tokenizer, f)
357 with open("semai_dna_label_encoder.pkl", "wb") as f:
358     pickle.dump(le, f)
359
360 """# BILSTM"""
361
362 !pip uninstall -y jax jaxlib
363 !pip install --upgrade tensorflow
364
365 import tensorflow as tf
366
367 vocab_size = len(tokenizer.word_index) + 1
368
369 model = tf.keras.Sequential([
370     tf.keras.layers.Embedding(input_dim=vocab_size, output_dim=128),
371     tf.keras.layers.Bidirectional(tf.keras.layers.LSTM(64, return_sequences=True)),
372     tf.keras.layers.GlobalMaxPooling1D(),
373     tf.keras.layers.Dense(64, activation='relu'),
374     tf.keras.layers.Dense(y_cat.shape[1], activation='softmax')
375 ])
376
377 model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
378 model.summary()
379
380 from tensorflow.keras.callbacks import EarlyStopping
381 import matplotlib.pyplot as plt
382 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
383 import numpy as np
384
385 early_stop = EarlyStopping(monitor='val_loss', patience=3, restore_best_weights=True)
386
387 history = model.fit(
388     X_train, y_train,
389     validation_split=0.2,
390     epochs=10,
391     batch_size=32,
392     callbacks=[early_stop]
393 )
394
395 plt.figure(figsize=(12, 4))
396
397 # Accuracy
398 plt.subplot(1, 2, 1)
399 plt.plot(history.history['accuracy'], label="Train")
400 plt.plot(history.history['val_accuracy'], label="Val")
401 plt.title("BiLSTM Accuracy")
402 plt.legend()
403
404 # Loss
405 plt.subplot(1, 2, 2)
406 plt.plot(history.history['loss'], label="Train")
407 plt.plot(history.history['val_loss'], label="Val")
408 plt.title("BiLSTM Loss")
409 plt.legend()
410
411 plt.tight_layout()
412 plt.show()
413

```

```

414 loss, acc = model.evaluate(X_test, y_test)
415 print(f"    BiLSTM Test Accuracy: {acc:.4f}")
416
417 from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay
418 import numpy as np
419
420 y_pred = model.predict(X_test)
421 y_pred_classes = np.argmax(y_pred, axis=1)
422 y_true = np.argmax(y_test, axis=1)
423
424 cm = confusion_matrix(y_true, y_pred_classes)
425 disp = ConfusionMatrixDisplay(confusion_matrix=cm, display_labels=le.classes_)
426 disp.plot(xticks_rotation=90)
427 plt.title("BiLSTM Confusion Matrix")
428 plt.tight_layout()
429 plt.show()
430
431 import numpy as np
432 from sklearn.metrics.pairwise import cosine_similarity
433
434 def test_sample(idx=None, num_reads=5, k=6):
435     print("="*60)
436
437     # Random sample or fixed index from test set
438     if idx is None:
439         idx = np.random.randint(len(X_test))
440
441     # Reconstruct original label and sequence
442     y_true_class = np.argmax(y_test[idx])
443     true_label = le.inverse_transform([y_true_class])[0]
444
445     # Find corresponding sequence (works if df_clean aligns with X_test)
446     # If not, you may need to store 'clean_sequences' as a lookup
447     sequence_df = pd.read_csv("semmai_dna_noisy_kmer_dataset.csv")
448     clean_seq = sequence_df.iloc[idx]["sequence"]
449
450     print(f"    Sample #{idx}")
451     print(f"    True Label: {true_label}")
452     print(f"    Clean Sequence: {clean_seq[:80]}{'...' if len(clean_seq) > 80 else ''}")
453     print("-"*60)
454
455     # Generate noisy reads
456     noisy_reads = [simulate_ids_errors(clean_seq) for _ in range(num_reads)]
457     print("    Noisy Reads and Predictions:")
458
459     best_sim = -1
460     best_read = ""
461     best_pred_label = ""
462
463     for r in noisy_reads:
464         try:
465             # Tokenize + encode for model
466             kmers = [r[i:i+k] for i in range(len(r) - k + 1)]
467             kmer_string = " ".join(kmers)
468             seq_encoded = tokenizer.texts_to_sequences([kmer_string])
469             X_input = pad_sequences(seq_encoded, maxlen=512, padding='post')
470
471             # Predict label
472             pred_probs = model.predict(X_input)
473             pred_class = np.argmax(pred_probs)
474             pred_label = le.inverse_transform([pred_class])[0]
475
476             # Semantic similarity
477             sim = cosine_similarity([dnabert_embed(clean_seq)], [dnabert_embed(r)])[0][0]
478
479             # Show
480             print(f"        {r[:60]}{'...' if len(r) > 60 else ''}")
481             print(f"        Predicted: {pred_label} | CosSim: {sim:.4f}")
482             if sim > best_sim:
483                 best_sim = sim
484                 best_read = r
485                 best_pred_label = pred_label
486         except Exception as e:
487             print(f"        Skipped read due to error: {e}")
488             continue
489
490     print("="*60)
491     print("    Best Read by Semantic Similarity:")
492     print(f"    {best_read[:80]}{'...' if len(best_read) > 80 else ''}")
493     print(f"    Predicted Label: {best_pred_label}")
494     print(f"    Matches True Label? {' ' if best_pred_label == true_label else ' '}")
495     print(f"    Cosine Similarity: {best_sim:.4f}")
496     print("="*60)

```

```

497
498 test_sample()                # Random test sample
499 test_sample(idx=42)          # Fixed index if you want reproducibility
500
501 lengths = [len(seq) for seq in df['sequence']]
502 import matplotlib.pyplot as plt
503 plt.hist(lengths, bins=50)
504 plt.title("Distribution of Sequence Lengths")
505 plt.xlabel("Length")
506 plt.ylabel("Count")
507 plt.show()
508
509 import pandas as pd
510 import numpy as np
511 from sklearn.model_selection import train_test_split
512 from sklearn.preprocessing import LabelEncoder
513 import tensorflow as tf
514 from tensorflow.keras import layers, models
515 from tensorflow.keras.preprocessing.text import Tokenizer
516 from tensorflow.keras.preprocessing.sequence import pad_sequences
517
518 # === Load dataset ===
519 df = pd.read_csv("semai_dna_noisy_kmer_dataset.csv")
520
521 # === Tokenize the noisy k-mer sequences ===
522 tokenizer = Tokenizer(char_level=False, oov_token="<OOV>")
523 tokenizer.fit_on_texts(df['noisy_sequence'])
524
525 sequences = tokenizer.texts_to_sequences(df['noisy_sequence'])
526 maxlen = 512
527 X = pad_sequences(sequences, maxlen=maxlen, padding='post')
528
529 # === Encode labels ===
530 label_encoder = LabelEncoder()
531 y = label_encoder.fit_transform(df['label'])
532 num_classes = len(label_encoder.classes_)
533 y_cat = tf.keras.utils.to_categorical(y, num_classes)
534
535 # === Split into train/test ===
536 X_train, X_test, y_train, y_test = train_test_split(X, y_cat, test_size=0.2, random_state=42)
537
538 # === Define CNN model ===
539 vocab_size = len(tokenizer.word_index) + 1
540
541 model = models.Sequential([
542     layers.Embedding(input_dim=vocab_size, output_dim=128),
543     layers.Conv1D(64, 5, activation='relu'),
544     layers.GlobalMaxPooling1D(),
545     layers.Dense(64, activation='relu'),
546     layers.Dense(num_classes, activation='softmax')
547 ])
548
549 model.compile(optimizer='adam', loss='categorical_crossentropy', metrics=['accuracy'])
550 model.summary()
551
552 # === Train ===
553 history = model.fit(X_train, y_train, validation_split=0.2, epochs=10, batch_size=16)
554
555 # === Evaluate ===
556 loss, acc = model.evaluate(X_test, y_test)
557 print(f"\n Test Accuracy: {acc:.4f}")
558
559 import pickle
560 # === Save model ===
561 model.save("semai_dna_cnn_model.keras")
562 print(" Saved model: semai_dna_cnn_model.keras")
563
564 # === Save tokenizer ===
565 with open("semai_dna_tokenizer.pkl", "wb") as f:
566     pickle.dump(tokenizer, f)
567 print(" Saved tokenizer: semai_dna_tokenizer.pkl")
568
569 # === Save label encoder ===
570 with open("semai_dna_label_encoder.pkl", "wb") as f:
571     pickle.dump(label_encoder, f)
572 print(" Saved label encoder: semai_dna_label_encoder.pkl")
573
574 #load the model
575 from tensorflow.keras.models import load_model
576 import pickle
577
578 model = load_model("semai_dna_cnn_model.keras")
579

```

```

580 with open("semai_dna_tokenizer.pkl", "rb") as f:
581     tokenizer = pickle.load(f)
582
583 with open("semai_dna_label_encoder.pkl", "rb") as f:
584     label_encoder = pickle.load(f)
585
586 from sklearn.metrics import classification_report, confusion_matrix
587 import seaborn as sns
588 import matplotlib.pyplot as plt
589 import numpy as np
590 from sklearn.model_selection import train_test_split
591 from sklearn.preprocessing import LabelEncoder
592 import tensorflow as tf
593 from tensorflow.keras import layers, models
594 from tensorflow.keras.preprocessing.text import Tokenizer
595 from tensorflow.keras.preprocessing.sequence import pad_sequences
596
597 # === Load dataset ===
598 df = pd.read_csv("semai_dna_noisy_kmer_dataset.csv")
599
600 # === Tokenize the noisy k-mer sequences ===
601 tokenizer = Tokenizer(char_level=False, oov_token="<OOV>")
602 tokenizer.fit_on_texts(df['noisy_sequence'])
603
604 sequences = tokenizer.texts_to_sequences(df['noisy_sequence'])
605 maxlen = 512
606 X = pad_sequences(sequences, maxlen=maxlen, padding='post')
607
608 # === Encode labels ===
609 label_encoder = LabelEncoder()
610 y = label_encoder.fit_transform(df['label'])
611 num_classes = len(label_encoder.classes_)
612 y_cat = tf.keras.utils.to_categorical(y, num_classes)
613
614 # === Split into train/test ===
615 X_train, X_test, y_train, y_test = train_test_split(X, y_cat, test_size=0.2, random_state=42)
616
617 # === 1. Predict test set ===
618 y_pred_probs = model.predict(X_test)
619 y_pred = np.argmax(y_pred_probs, axis=1)
620 y_true = np.argmax(y_test, axis=1)
621 #
622 # === 2. Decode labels ===
623 class_names = label_encoder.classes_
624
625 # === 3. Classification Report ===
626 print("      Classification Report:")
627 #print(classification_report(y_true, y_pred, target_names=class_names))
628
629 # === 4. Confusion Matrix ===
630 cm = confusion_matrix(y_true, y_pred)
631
632 plt.figure(figsize=(12, 8))
633 sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', xticklabels=class_names, yticklabels=class_names)
634 plt.title("Confusion Matrix")
635 plt.xlabel("Predicted Label")
636 plt.ylabel("True Label")
637 plt.tight_layout()
638 plt.show()
639
640 """# DNABERT"""
641
642 !pip install triton
643
644 from transformers import AutoTokenizer, AutoModel
645 import torch
646
647 device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
648
649 model_id = "zhihan1996/DNA_bert_6"
650 tokenizer = AutoTokenizer.from_pretrained(model_id)
651 model = AutoModel.from_pretrained(model_id).to(device)
652 model.eval()
653
654 def get_dnabert6_embedding(seq):
655     kmer_seq = to_kmers(seq.upper())
656     inputs = tokenizer(kmer_seq, return_tensors="pt").to(device)
657     with torch.no_grad():
658         outputs = model(**inputs)
659         cls_vec = outputs.last_hidden_state[:, 0, :] # [CLS] token
660         return cls_vec.squeeze().cpu().numpy() # shape: (768,)
661
662 def to_kmers(seq, k=6):

```



```

663     return " ".join([seq[i:i+k] for i in range(len(seq) - k + 1)])
664
665 import pandas as pd
666 import numpy as np
667
668 df = pd.read_csv("semai_dna_noisy_kmer_dataset.csv")
669
670 embeddings, labels = [], []
671
672 for seq, label in zip(df['sequence'], df['label']):
673     try:
674         vec = get_dnabert6_embedding(seq[:300]) # truncate for efficiency
675         embeddings.append(vec)
676         labels.append(label)
677     except Exception as e:
678         print(f"Skipped due to error: {e}")
679
680 np.save("dnabert6_embeddings.npy", np.stack(embeddings))
681 pd.Series(labels).to_csv("dnabert6_labels.csv", index=False)

```

Listing 1. DeepJSOC Pipeline – Implemented by Eslim

```

1  # -*- coding: utf-8 -*-
2  """NLP_Project.ipynb
3
4  Automatically generated by Colab.
5
6  Original file is located at
7  https://colab.research.google.com/drive/1xBirMkDp0h01X8N3_JyE0FEAfr2sEK1L
8  """
9
10 from google.colab import drive
11 drive.mount('/content/drive')
12
13 !wget ftp://ftp.ensembl.org/pub/release-108/fasta/homo_sapiens/dna/Homo_sapiens.GRCh38.dna.primary_assembly.fa.gz
14 !gunzip Homo_sapiens.GRCh38.dna.primary_assembly.fa.gz
15
16 from Bio import Entrez, SeqIO
17 import pathlib
18
19 Entrez.email = "eslimranae@gmail.com"
20
21 mt_id = "NC_012920.1"
22 out_dir = pathlib.Path("/content/drive/MyDrive/NLP_Project/dna_jsoc")
23 out_file = out_dir / "human_mt.fasta"
24 out_dir.mkdir(exist_ok=True, parents=True)
25
26 with Entrez.efetch(db="nucleotide", id=mt_id, rettype="fasta", retmode="text") as handle:
27     seq_record = SeqIO.read(handle, "fasta")
28
29 SeqIO.write(seq_record, out_file, "fasta")
30 print(f"Downloaded human mitochondrial genome ({len(seq_record.seq)} bp) to\n {out_file}")
31
32 !pip uninstall -y triton
33
34 !pip install --quiet torch torchvision torchaudio --index-url https://download.pytorch.org/whl/cul21
35 !pip install --quiet biopython sacrebleu rouge-score tqdm
36
37 !pip install --quiet vector-quantize-pytorch
38
39 # Commented out IPython magic to ensure Python compatibility.
40 # %cd /content/drive/MyDrive/NLP_Project/deepjsoc
41
42 !python train_estimator.py \
43     --channel_bits 128 \
44     --Nc 2 \
45     --d_rnn 256 \
46     --bs 64 \
47     --epochs 6 \
48     --step 200 \
49     --training_Pd 0.03 \
50     --training_Ps 0.03 \
51     --training_Pi 0.03
52
53 import shutil, glob, pathlib
54 src = glob.glob('/content/drive/MyDrive/NLP_Project/deepjsoc/*estimator*.pth')[0]
55
56 OUT_DIR = pathlib.Path('/content/drive/MyDrive/NLP_Project/dna_jsoc')
57 OUT_DIR.mkdir(exist_ok=True)
58
59 dst = OUT_DIR / 'estimator.pt'
60 shutil.copy(src, dst)
61 print('Estimator saved ', dst)

```

```

62
63 from Bio import SeqIO
64 import random
65 import pathlib, json
66
67 SEQ_LEN = 256
68 RAW_FASTA = '/content/drive/MyDrive/NLP_Project/sequence.fasta'
69 OUT_DIR = pathlib.Path('/content/drive/MyDrive/NLP_Project/dna_jsoc')
70 OUT_DIR.mkdir(exist_ok=True)
71
72 seq = ''.join(str(rec.seq).upper() for rec in SeqIO.parse(RAW_FASTA, 'fasta'))
73
74 chunks = [seq[i:i+SEQ_LEN] for i in range(0, len(seq)-SEQ_LEN, SEQ_LEN)]
75 random.shuffle(chunks)
76
77 a, b = int(.8*len(chunks)), int(.9*len(chunks))
78 splits = {'train':chunks[:a], 'val':chunks[a:b], 'test':chunks[b:]}
79
80 for split, data in splits.items():
81     with open(OUT_DIR/f'chimp256.{split}', 'w') as f:
82         f.write('\n'.join(data))
83 print({k:len(v) for k, v in splits.items()})
84
85 from Bio import SeqIO
86 import random
87 import pathlib
88
89 SEQ_LEN = 256
90 CHIMP_FASTA = '/content/drive/MyDrive/NLP_Project/sequence.fasta'
91 HUMAN_FASTA = '/content/drive/MyDrive/NLP_Project/deepjsoc/Homo_sapiens.GRCh38.dna.primary_assembly.fa'
92 OUT_DIR = pathlib.Path('/content/drive/MyDrive/NLP_Project/dna_jsoc')
93 OUT_DIR.mkdir(exist_ok=True)
94
95 def load_seq(fasta_path: str) -> str:
96     """Read a FASTA and return one uppercase string of all records concatenated."""
97     return ''.join(str(rec.seq).upper() for rec in SeqIO.parse(fasta_path, 'fasta'))
98
99 chimp_seq = load_seq(CHIMP_FASTA)
100 human_seq = load_seq(HUMAN_FASTA)
101
102 combined_seq = chimp_seq + human_seq
103 print(f"Total length: {len(chimp_seq)} (chimp) + {len(human_seq)} (human) = {len(combined_seq)} bases")
104
105 chunks = [
106     combined_seq[i : i + SEQ_LEN]
107     for i in range(0, len(combined_seq) - SEQ_LEN + 1, SEQ_LEN)
108 ]
109
110 random.shuffle(chunks)
111
112 a = int(0.8 * len(chunks))
113 b = int(0.9 * len(chunks))
114 splits = {
115     'train': chunks[:a],
116     'val':   chunks[a:b],
117     'test':  chunks[b:]
118 }
119
120 for split, data in splits.items():
121     filename = f"chimp{SEQ_LEN}.{split}"
122     out_path = OUT_DIR / filename
123     with open(out_path, 'w') as f:
124         f.write('\n'.join(data))
125     print(f"Wrote {len(data)} sequences to {out_path}")
126
127 print({k: len(v) for k, v in splits.items()})
128
129 from Bio import SeqIO
130 import random, pathlib
131
132 SEQ_LEN = 256
133 CHIMP_FASTA = '/content/drive/MyDrive/NLP_Project/sequence.fasta'
134 HUMAN_FASTA = '/content/drive/MyDrive/NLP_Project/dna_jsoc/human_mt.fasta'
135 OUT_DIR = pathlib.Path('/content/drive/MyDrive/NLP_Project/dna_jsoc')
136 OUT_DIR.mkdir(exist_ok=True)
137
138 def load_seq(path):
139     return ''.join(str(rec.seq).upper() for rec in SeqIO.parse(path, 'fasta'))
140
141 chimp = load_seq(CHIMP_FASTA)
142 human = load_seq(HUMAN_FASTA)
143 combined = chimp + human
144 print(f"Lengths      chimp: {len(chimp)} bp, human-mt: {len(human)} bp, total: {len(combined)} bp")

```

```

145
146 chunks = [combined[i:i+SEQ_LEN] for i in range(0, len(combined)-SEQ_LEN+1, SEQ_LEN)]
147 random.shuffle(chunks)
148 a, b = int(.8*len(chunks)), int(.9*len(chunks))
149 splits = {
150     'train': chunks[:a],
151     'val':   chunks[a:b],
152     'test':  chunks[b:]
153 }
154
155 for split, data in splits.items():
156     path = OUT_DIR / f"chimp{SEQ_LEN}.{split}"
157     with open(path, 'w') as f:
158         f.write('\n'.join(data))
159     print(f"Wrote {len(data)} seqs to {path}")
160
161 import json, pathlib
162
163 tok2idx = {
164     "<PAD>": 0,
165     "<START>": 1,
166     "<END>": 2,
167     "<MASK>": 3,
168     "<UNK>": 4,
169     "A": 5,
170     "C": 6,
171     "G": 7,
172     "T": 8,
173     "N": 9
174 }
175 idx2tok = [k for k,_ in sorted(tok2idx.items(), key=lambda kv: kv[1])]
176
177 vocab = {"token_to_idx": tok2idx,
178         "idx_to_token": idx2tok}
179
180 path = pathlib.Path('/content/drive/MyDrive/NLP_Project/dna_jsoc/vocab.json')
181 with open(path, 'w') as f:
182     json.dump(vocab, f, indent=2)
183
184 print('    Vocabulary now contains <PAD>, <START>, <END>, <MASK>, <UNK> and A/C/G/T/N.')
185
186 !pip install w3lib
187
188 import torch
189 ckpt = torch.load('/content/drive/MyDrive/NLP_Project/dna_jsoc/estimator.pt')
190 w = ckpt['bir_layers.0.weight_ih_l0']
191 print('actual_size in checkpoint =', w.shape[1])
192
193 !python main.py \
194     --vocab-file /content/drive/MyDrive/NLP_Project/dna_jsoc/vocab.json \
195     --batch-size 4 --epochs 6 --warm-start -1 \
196     --safety-len 360 \
197     --marker-enc-size 360 \
198     --channel-in-len 36 \
199     --estimator-file /content/drive/MyDrive/NLP_Project/dna_jsoc/estimator.pt
200
201 !pip install python-Levenshtein
202
203 !python performance.py \
204     --vocab-file /content/drive/MyDrive/NLP_Project/dna_jsoc/vocab.json \
205     --checkpoint-path checkpoints \
206     --batch-size 8 \
207     --ps 0.03 \
208     --pd-list 0.0 0.02 0.05 0.08 \
209     --estimator-file /content/drive/MyDrive/NLP_Project/dna_jsoc/estimator.pt
210
211 import pandas as pd
212
213 df = pd.read_csv('/content/drive/MyDrive/NLP_Project/dna_jsoc/results.csv')
214
215 for col in ['Pd', 'BLEU-1', 'DNABERT-sim', 'Edit-Distance', 'Token-Accuracy', 'Sequence-Accuracy']:
216     df[col] = pd.to_numeric(df[col], errors='coerce')
217
218 print("\n===== Performance Metrics =====")
219 print(f"{'Pd':<8} {'BLEU-1':<10} {'DNABERT-sim':<14} {'EditDist':<12} {'TokenAcc':<12} {'SeqAcc':<10}")
220 print("="*70)
221 for _, row in df.iterrows():
222     print(f"{row['Pd']:<8.3f} {row['BLEU-1']:<10.4f} {row['DNABERT-sim']:<14.4f} "
223           f"{row['Edit-Distance']:<12.2f} {row['Token-Accuracy']:<12.4f} {row['Sequence-Accuracy']:<10.4f}")
224
225 import pandas as pd, matplotlib.pyplot as plt
226 df.plot(x='Pd', y=['BLEU-1', 'BERT-sim'], marker='o')
227 plt.xlabel('Insertion / Deletion / Substitution probability (Pd)')

```

```

228 plt.title('DeepJSOC on chimp-256')
229 plt.show()
230
231 import pandas as pd
232 import matplotlib.pyplot as plt
233
234 df = pd.read_csv('/content/drive/MyDrive/NLP_Project/dna_jsoc/results.csv')
235
236 plt.plot(df['Pd'], df['BLEU-1'], marker='o', label='BLEU-1')
237 plt.xlabel('Insertion / Deletion / Substitution probability (Pd)')
238 plt.ylabel('BLEU Score')
239 plt.title('BLEU-1 vs. Pd')
240 plt.grid(True)
241 plt.legend()
242 plt.show()
243
244 import pandas as pd
245 import matplotlib.pyplot as plt
246
247 df = pd.read_csv('/content/drive/MyDrive/NLP_Project/dna_jsoc/results.csv')
248
249 plt.figure(figsize=(10, 6))
250 plt.plot(df['Pd'], df['BLEU-1'], marker='o', label='BLEU-1')
251 plt.plot(df['Pd'], df['BERT-sim'], marker='o', label='BERT-sim')
252 plt.xlabel('Insertion / Deletion / Substitution probability (Pd)')
253 plt.ylabel('Score')
254 plt.title('BLEU-1 vs BERT-sim across different Pd levels')
255 plt.legend()
256 plt.grid(True)
257 plt.tight_layout()
258 plt.show()
259
260 print(df.columns)
261
262 # Ensure numeric types
263 df['Pd'] = pd.to_numeric(df['Pd'], errors='coerce')
264 df['BLEU-1'] = pd.to_numeric(df['BLEU-1'], errors='coerce')
265 df['BERT-sim'] = pd.to_numeric(df['BERT-sim'], errors='coerce')
266 df['EditDist'] = pd.to_numeric(df['Edit-Distance'], errors='coerce')
267
268 print("\n===== Performance Metrics =====")
269 print(f"{'Pd':<8} {'BLEU-1':<10} {'BERT-sim':<12} {'EditDist':<10}")
270 print("="*44)
271 for _, row in df.iterrows():
272     print(f"{row['Pd']:<8.3f} {row['BLEU-1']:<10.4f} {row['BERT-sim']:<12.4f} {row['EditDist']:<10.4f}")
273
274 !pip install transformers

```

Listing 2. SemAI-DNA Pipeline – Implemented by Betül

```

1 # %% [markdown]
2 # Installs the only external libraries we need.
3 # -q keeps the notebook output tidy.
4
5 # %%
6 %pip install -q biopython sympy
7
8 # %% [markdown]
9 # All imports live in one cell so dependencies are obvious.
10 # random.seed(42) makes every run reproducible.
11 # Suppressing warnings avoids clutter when Biopython prints remarks.
12
13 # %%
14 import random, warnings, textwrap
15 from pathlib import Path
16 from collections import Counter
17 from difflib import SequenceMatcher
18
19 from Bio import SeqIO
20 from Bio.Align import PairwiseAligner
21 from sympy.ntheory.modular import solve_congruence
22
23 # reproducibility
24 random.seed(42)
25 try:
26     import numpy as np
27     np.random.seed(42)
28 except ImportError:
29     pass # numpy will be installed later for metrics
30
31 warnings.filterwarnings("ignore")
32
33 # %%

```

```

34 # == DNA / RRNS constants =====
35 BASE_MAP = {'A': 0, 'C': 1, 'G': 2, 'T': 3}
36 INV_MAP = {v: k for k, v in BASE_MAP.items()}
37
38 MODULI = [7,11,13,17,19] # 3 information + 2 redundant residues
39 BLOCK_LEN = 12 # 12 bases one RRNS block
40 SEG_LEN = 100 # 100-bp segments used for experiments
41
42 # Shared tuples for random-error generator
43 BASES = tuple(BASE_MAP.keys())
44 OTHER = {b: tuple(x for x in BASES if x != b) for b in BASES}
45 # =====
46
47
48 #
49
50 def read_fasta_sequences(file_path: str, segment_len: int = SEG_LEN) -> list[str]:
51     """
52     Load a FASTA file and (optionally) slice each record into fixed-length segments.
53
54     Parameters
55     -----
56     file_path : str
57         Path to the FASTA file.
58     segment_len : int, default SEG_LEN
59         If >0, cut every record into non-overlapping chunks of this length.
60
61     Returns
62     -----
63     list[str]
64         Upper-case DNA segments; empty if no sequence met the length criterion.
65
66     Raises
67     -----
68     FileNotFoundError
69         If 'file_path' does not exist.
70     """
71     file_path = Path(file_path)
72     if not file_path.exists():
73         raise FileNotFoundError(f"{file_path} not found")
74
75     segments: list[str] = []
76     for rec in SeqIO.parse(file_path, "fasta"):
77         seq = str(rec.seq).upper()
78         if segment_len:
79             for i in range(0, len(seq), segment_len):
80                 chunk = seq[i : i + segment_len]
81                 if len(chunk) == segment_len:
82                     segments.append(chunk)
83             else:
84                 segments.append(seq)
85     return segments
86
87 #
88
89 def introduce_ids_errors(
90     seq: str,
91     rates: dict = {"ins": 0.02, "del": 0.02, "sub": 0.02},
92 ) -> str:
93     """
94     Introduce random Insertion / Deletion / Substitution (IDS) errors.
95
96     Randomness is deterministic if 'random.seed()' was set earlier.
97
98     Parameters
99     -----
100     seq : str
101         Original (clean) DNA sequence.
102     rates : dict
103         Error probabilities; keys = 'ins', 'del', 'sub'.
104
105     Returns
106     -----
107     str
108         Corrupted sequence containing insertions, deletions, and substitutions.
109     """
110     out: list[str] = []
111     i = 0
112     r_ins, r_del, r_sub = rates["ins"], rates["del"], rates["sub"]
113     while i < len(seq):
114         rnd = random.random()

```

```

115         if rnd < r_ins:                                # insertion
116             out.append(random.choice(BASES))
117             out.append(seq[i])
118             i += 1
119         elif rnd < r_ins + r_del:                        # deletion (skip one base)
120             i += 1
121         elif rnd < r_ins + r_del + r_sub:                # substitution
122             out.append(random.choice(OTHER[seq[i]]))
123             i += 1
124         else:                                            # no error
125             out.append(seq[i])
126             i += 1
127     return "".join(out)
128
129 # %%
130 from functools import reduce
131
132 # --- DNA      numeric -----
133 # case-insensitive base numeric
134 to_base4 = lambda seq: [BASE_MAP[b.upper()] for b in seq]
135
136 def nums_to_int(block: list[int]) -> int:
137     """Convert base-4 digits to an integer via left-shift (x4)."""
138     return reduce(lambda acc, d: (acc << 2) + d, block, 0)
139
140
141 # --- RRNS encode -----
142 def rrns_encode_block(
143     block_digits: list[int],
144     moduli: list[int] = MODULI
145 ) -> list[int]:
146     n = nums_to_int(block_digits)
147     return [n % m for m in moduli]
148
149 def rrns_encode_sequence(
150     seq: str,
151     moduli: list[int] = MODULI,
152     block_len: int = BLOCK_LEN
153 ) -> list[list[int]]:
154     """
155     Split 'seq' into base-4 digits, chunk into 'block_len'-sized blocks,
156     then RRNS-encode each block.
157     """
158     digits = to_base4(seq)
159     # drop any partial final block shorter than block_len
160     blocks = [digits[i : i + block_len]
161               for i in range(0, len(digits), block_len)
162               if len(digits[i : i + block_len]) == block_len]
163     return [rrns_encode_block(b, moduli) for b in blocks]
164
165
166 # --- load & preview data -----
167 segments = read_fasta_sequences("sample.fasta")
168 print(f"Loaded {len(segments)} segments")
169
170 original = segments[0]
171 print("Original (first 80 bp):", textwrap.shorten(original, width=80))
172
173
174 # --- simulate IDS errors -----
175 corrupted = introduce_ids_errors(original)
176 print("Corrupted (first 80 bp):", textwrap.shorten(corrupted, width=80))
177
178
179 # --- RRNS encode a prefix -----
180 encoded_blocks = rrns_encode_sequence(corrupted[:500])
181 print("First RRNS block residues:", encoded_blocks[0])
182
183
184 # --- multi-read generation (immutable default rates) -----
185 def generate_multi_reads(
186     sequence: str,
187     num_reads: int = 5,
188     rates: dict | None = None
189 ) -> list[str]:
190     """
191     Return 'num_reads' corrupted copies of 'sequence'.
192
193     Parameters
194     -----
195     sequence : str
196         Clean DNA to corrupt.
197     num_reads : int

```

```

198         How many simulated reads to produce.
199         rates : dict, optional
200         IDS error probabilities; keys 'ins','del','sub'. Defaults to 2% each.
201         """
202         if rates is None:
203             rates = {"ins": 0.02, "del": 0.02, "sub": 0.02}
204         return [introduce_ids_errors(sequence, rates) for _ in range(num_reads)]
205
206 multi_reads = generate_multi_reads(corrupted[:SEG_LEN], num_reads=10)
207 for idx, r in enumerate(multi_reads[:3], 1):
208     print(f"Read {idx}:", textwrap.shorten(r, width=80))
209
210
211 # --- quick pairwise align -----
212 def align_reads_pairwise(reads: list[str]) -> list[str]:
213     """
214     Align all reads to the first read (global alignment).
215     Gaps ( - ) inserted relative to the template.
216     """
217     if len(reads) < 2:
218         return reads
219
220     aligner = PairwiseAligner()
221     aligner.mode = "global"
222     template = reads[0]
223     aligned_reads = [template]
224
225     for read in reads[1:]:
226         aln = aligner.align(template, read)[0]
227         aligned_reads.append(aln.target)
228     return aligned_reads
229
230 aligned_reads = align_reads_pairwise(multi_reads)
231 for idx, r in enumerate(aligned_reads[:3], 1):
232     print(f"Aligned {idx}:", textwrap.shorten(r, width=80))
233
234 # %%
235 # --- majority vote -----
236 def majority_vote(aligned_reads: list[str]) -> str:
237     """
238     Build a consensus sequence by column-wise majority voting.
239     Gaps '-' are ignored.
240     """
241     if not aligned_reads:
242         return ""
243
244     length = len(aligned_reads[0])
245     consensus = []
246     for i in range(length):
247         column = [r[i] for r in aligned_reads if i < len(r) and r[i] != "-"]
248         if column:
249             common = Counter(column).most_common(1)[0][0]
250             consensus.append(common)
251     return "".join(consensus)
252
253 # %%
254 # build consensus from the previously aligned reads
255 reconstructed = majority_vote(aligned_reads).upper()
256 print("Consensus (80 bp):", textwrap.shorten(reconstructed, width=80))
257
258 # similarity to the original segment
259 sim = SequenceMatcher(None, original[: len(reconstructed)], reconstructed).ratio()
260 print(f"Similarity to original: {sim:.2%}")
261
262 # %%
263 # --- RRNS decode -----
264 def rrns_decode_block(residues: list[int], moduli=MODULI) -> int | None:
265     """
266     Recover X from residues using CRT; return None if residues are inconsistent.
267     """
268     congr = [(residues[i], moduli[i]) for i in range(len(moduli))]
269     try:
270         x, _ = solve_congruence(*congr)
271         return int(x)
272     except ValueError:
273         # no solution
274         return None
275
276 def int_to_dna(n: int, length: int = BLOCK_LEN) -> str:
277     """Convert integer back to fixed-length DNA (base-4 decoding)."""
278     digits = [(n >> (2 * k)) & 3 for k in range(length)][::-1]
279     return "".join(INV_MAP[d] for d in digits)
280

```

```

281 # %%
282 # decode the first RRNS block generated earlier
283 decoded_int = rrns_decode_block(encoded_blocks[0])
284 decoded_dna = int_to_dna(decoded_int) if decoded_int is not None else "ERR"
285
286 print("Decoded DNA block :", decoded_dna)
287 print("Original 8 bp      :", original[:BLOCK_LEN])
288
289 # %%
290 # 1) Load and segment FASTA
291 segments = read_fasta_sequences("sample.fasta", segment_len=SEG_LEN)
292 print(f"      Total segments = {len(segments)}")
293 print("      Example segment:", textwrap.shorten(segments[0], width=80))
294
295 # 2) RRNS-encode the clean segment
296 encoded_blocks = rrns_encode_sequence(segments[0])
297 print(f"\ n      RRNS encoding complete      {len(encoded_blocks)} blocks")
298 print("First block residues:", encoded_blocks[0])
299
300 # %%
301 # 3) Generate corrupted multi-reads and build consensus
302 segment      = segments[0]
303 multi_reads  = generate_multi_reads(segment, num_reads=5)
304 aligned      = align_reads_pairwise(multi_reads)
305 consensus    = majority_vote(aligned)
306
307 print("\ n      Original segment :", textwrap.shorten(segment, width=80))
308 print("      Consensus      :", textwrap.shorten(consensus, width=80))
309
310 # %%
311 from difflib import SequenceMatcher
312
313 # 4) Marker helpers
314 LONG_START_MARKER = "ACGTACGTACGT"
315 LONG_END_MARKER   = "GTACGTACGTAC"
316
317 def add_markers(segment: str,
318                 start_marker: str = LONG_START_MARKER,
319                 end_marker : str = LONG_END_MARKER) -> str:
320     return f"{start_marker}{segment}{end_marker}"
321
322
323 def validate_marker_strict(seq: str,
324                            start_marker: str = LONG_START_MARKER,
325                            end_marker : str = LONG_END_MARKER) -> bool:
326     seq = seq.upper()
327     return seq.startswith(start_marker) and seq.endswith(end_marker)
328
329 def similarity(a: str, b: str) -> float:
330     """Return SequenceMatcher ratio in [0,1]."""
331     return SequenceMatcher(None, a, b).ratio()
332
333
334 def validate_marker_fuzzy(sequence: str,
335                            start_marker: str = LONG_START_MARKER,
336                            end_marker : str = LONG_END_MARKER,
337                            threshold: float = 0.85) -> bool:
338     seq = sequence.upper()
339     start = seq[:len(start_marker)]
340     end = seq[-len(end_marker):]
341     return (
342         similarity(start, start_marker) >= threshold and
343         similarity(end, end_marker) >= threshold
344     )
345
346 # %%
347 # 4a) Test marker protection end-to-end
348 marked      = add_markers(segment)
349 corrupted_mark = introduce_ids_errors(marked)
350 print("\ n      Marked + corrupted:", textwrap.shorten(corrupted_mark, width=80))
351 print("      Strict valid? ", validate_marker_strict(corrupted_mark))
352 print("      Fuzzy valid?  ", validate_marker_fuzzy(corrupted_mark))
353
354 # %%
355 def test_fuzzy_marker_robustness(
356     segments: list[str],
357     num_samples: int = 1000,
358     threshold: float = 0.85,
359 ) -> None:
360     """
361     Randomly corrupt markerprotected segments and report
362     the fraction that still passes fuzzy validation.
363     """

```



```

364     detected = 0
365     # make sure we don't exceed the list
366     sample_list = segments[:min(num_samples, len(segments))]
367     for seg in sample_list:
368         marked = add_markers(seg) # attach markers
369         corrupted = introduce_ids_errors(marked) # corrupt
370         # use default start/end markers, override threshold
371         if validate_marker_fuzzy(
372             corrupted,
373             start_marker=LONG_START_MARKER,
374             end_marker=LONG_END_MARKER,
375             threshold=threshold
376         ):
377             detected += 1
378
379     total = len(sample_list)
380     pct = 100 * detected / total if total else 0.0
381     print(
382         f"        Fuzzy marker survival: {detected}/{total} "
383         f"({pct:.2f}% at thr={threshold})"
384     )
385
386 # %%
387 # 1) multi-read similarity scoring (re-uses 'similarity' helper)
388 def score_multi_reads(
389     reads: list[str],
390     reference: str,
391     top_n: int = 3
392 ) -> list[str]:
393     """
394     Rank reads by similarity to reference; return the best 'top_n'.
395     """
396     # reuse the same 'similarity' helper
397     ranked = sorted(
398         reads,
399         key=lambda r: similarity(r, reference),
400         reverse=True
401     )
402     return ranked[:top_n]
403
404 #
405
406 # Demo of top-N scoring + consensus + metrics
407 # -----
408
409 segment = segments[0]
410 multi_reads = generate_multi_reads(segment, num_reads=10)
411 top_reads = score_multi_reads(multi_reads, segment, top_n=3)
412
413 print("    Original:", textwrap.shorten(segment, width=80))
414 print("\n    Top-3 reads (by similarity):")
415 for i, r in enumerate(top_reads, 1):
416     print(f"    Read {i}:", textwrap.shorten(r, width=80))
417
418 # 2) align majority vote consensus
419 aligned = align_reads_pairwise(top_reads)
420 consensus = majority_vote(aligned)
421
422 # 3) evaluation metrics -----
423 def edit_distance(a: str, b: str) -> int:
424     """Simple Hamming+length diff."""
425     return sum(1 for x,y in zip(a,b) if x!=y) + abs(len(a)-len(b))
426
427 def normalized_edit_distance(a: str, b: str) -> float:
428     """Levenshtein-like normalized to [0,1]."""
429     mxd = max(len(a), len(b))
430     return edit_distance(a,b)/mxd if mxd else 0.0
431
432 def bit_error_rate(original: str, predicted: str) -> float:
433     """Fraction of mismatched bases over the longer sequence."""
434     mxd = max(len(original), len(predicted))
435     if mxd == 0:
436         return 0.0
437     errors = sum(1 for o,p in zip(original, predicted) if o!=p)
438     return errors / mxd
439
440 print("\n    Metrics on consensus vs. original")
441 print(f"    Edit distance : {edit_distance(segment, consensus)}")
442 print(f"    Edit rate      : {normalized_edit_distance(segment, consensus):.3f}")
443 print(f"    BER            : {bit_error_rate(segment, consensus):.3f}")
444
445 # %%

```

```

446 # ==== Cell 22      MSA helper =====
447 import subprocess
448 import tempfile
449 import shutil
450 from pathlib import Path
451 from Bio import AlignIO
452 from Bio.Align import PairwiseAligner
453
454 def run_msa(reads: list[str]) -> list[str]:
455     """
456     Align 'reads' with MAFFT if available, otherwise fall back to pairwise aligner.
457     Returns a list of gap-padded strings of equal length.
458     """
459     if shutil.which("mafft"):
460         with tempfile.TemporaryDirectory() as tmp:
461             in_fasta = Path(tmp) / "in.fasta"
462             out_fasta = Path(tmp) / "out.fasta"
463
464             # write input sequences to FASTA
465             with in_fasta.open("w") as fh:
466                 for i, r in enumerate(reads):
467                     fh.write(f">read{i}\n{r}\n")
468
469             # run MAFFT quietly
470             cmd = ["mafft", "--auto", "--quiet", str(in_fasta)]
471             subprocess.run(cmd, stdout=out_fasta.open("w"), check=True)
472
473             # read aligned output
474             aln = AlignIO.read(out_fasta, "fasta")
475             return ["".join(rec.seq) for rec in aln]
476
477     # fallback to pairwise alignment if MAFFT not found
478     print("MAFFT not found - using pairwise alignment")
479     return align_reads_pairwise(reads)
480
481 # %%
482 # ==== Cell 23      re-encode + CRT decode =====
483 def decode_consensus_with_rrns(consensus: str) -> str | None:
484     """
485     Split consensus into RRNS blocks, compute residues, CRT-decode each block.
486     Returns full DNA string on success, None if any block fails.
487     """
488     # normalize to uppercase
489     consensus = consensus.upper()
490     orig_len = len(consensus)
491
492     # pad so that length is a multiple of BLOCK_LEN
493     rem = orig_len % BLOCK_LEN
494     if rem:
495         consensus += "A" * (BLOCK_LEN - rem)
496
497     decoded = []
498     # process each fixed-size block
499     for i in range(0, len(consensus), BLOCK_LEN):
500         blk = consensus[i : i + BLOCK_LEN]
501         residues = rrns_encode_block(to_base4(blk))
502         x = rrns_decode_block(residues)
503         if x is None:
504             return None # uncorrectable block
505         decoded.append(int_to_dna(x))
506
507     full_decoded = "".join(decoded)
508     # strip padding to original length
509     return full_decoded[:orig_len]
510
511 # %%
512 # ==== Cell 24      payload extractor =====
513 def extract_payload_with_markers(seq: str) -> str | None:
514     """
515     Locate payload using long markers (fuzzy match). Returns inner DNA or None.
516     """
517     # always operate on uppercase
518     seq = seq.upper()
519     # reject if markers are not present even fuzzily
520     if not validate_marker_fuzzy(
521         seq,
522         start_marker=LONG_START_MARKER,
523         end_marker=LONG_END_MARKER,
524         threshold=0.85
525     ):
526         return None
527     # strip off the start/end markers
528     return seq[len(LONG_START_MARKER) : -len(LONG_END_MARKER)]

```

```

529
530 # %%
531 def mrs_filter(reads: list[str], w: int, K: int = 7) -> list[str]:
532     """
533     Multi-Reads Screening (MRS) per Luo et al.
534     Select the top-w most reliable reads out of v by:
535     1) computing Q[i,j] = count of base at pos j in all reads (penalty -v if gap)
536     2) q[i] = mean(Q[i,:]); select indices of top-w q's
537     3) if tie at the w-th, break ties by evaluating k-mer Q' similarly
538     Parameters
539     -----
540     reads : list[str]
541         List of v aligned reads (all same length).
542     w : int
543         Number of reads to select.
544     K : int, optional (default=7)
545         k-mer length for tie-breaking.
546     Returns
547     -----
548     list[str]
549         The selected w reads (in original order).
550     """
551     v = len(reads)
552     if v <= w:
553         return reads[:]
554
555     k = len(reads[0])
556     # 1) build Q matrix
557     Q = [[0]*k for _ in range(v)]
558     for j in range(k):
559         column = [reads[i][j] for i in range(v)]
560         counts = Counter(column)
561         for i in range(v):
562             base = reads[i][j]
563             if base == '-':
564                 Q[i][j] = -v
565             else:
566                 Q[i][j] = counts[base]
567     # 2) row means scores q
568     q = [sum(Q[i]) / k for i in range(v)]
569     # find the cutoff score
570     threshold = sorted(q, reverse=True)[w-1]
571     # initial top indices
572     top_idx = [i for i, score in enumerate(q) if score > threshold]
573     # those equal to threshold
574     ties = [i for i, score in enumerate(q) if score == threshold]
575     # need t of the ties to fill w
576     need = w - len(top_idx)
577     if need > 0 and ties:
578         # build k-mer Q' for tied indices
579         # first build k-mer subsequences
580         # sliding windows of length K over each read (ignoring gaps is optional)
581         # count occurrences per window position
582         km = k - K + 1
583         Qp = {i: [0]*km for i in ties}
584         for pos in range(km):
585             # collect each reads k-mer at pos
586             kmers = [reads[i][pos:pos+K] for i in ties]
587             counts = Counter(kmers)
588             for i, mer in zip(ties, kmers):
589                 Qp[i][pos] = counts[mer]
590         # compute mean q'
591         qp = {i: sum(Qp[i])/km for i in ties}
592         # sort ties by qp descending and take first 'need'
593         sel = sorted(qp, key=lambda i: qp[i], reverse=True)[:need]
594         top_idx += sel
595
596     # return reads in the same order as input
597     return [reads[i] for i in sorted(top_idx)]
598
599 # %%
600 # ==== Cell 24      payload extractor (updated) =====
601 def extract_payload_with_markers(
602     seq: str,
603     threshold: float = 0.75, # lowered default threshold
604 ) -> str | None:
605     """
606     Locate payload using long markers (fuzzy match). Returns inner DNA or None.
607     Uses validate_marker_fuzzy with given threshold.
608     """
609     # if fuzzymatch fails, give up
610     if not validate_marker_fuzzy(seq, LONG_START_MARKER, LONG_END_MARKER, threshold):
611         return None

```

```

612 # strip off the markers
613 return seq[len(LONG_START_MARKER):-len(LONG_END_MARKER)]
614
615 # %%
616 # ==== Cell 25      pipeline demo (with lenient marker) =====
617 def run_pipeline(
618     segment: str,
619     num_reads: int      = 8,
620     mode: str           = "rrns+msa",
621     ids_rates: dict     = {"ins":0.02,"del":0.02,"sub":0.02},
622 ) -> dict:
623     """
624     mode      {'rrns+msa', 'rrns+marker', 'rrns+msa+mrs'}
625     Returns dict with keys: success, BER, edit (or reason on failure).
626     """
627     clean = segment
628
629     # wrap in markers if requested
630     if "marker" in mode:
631         segment = add_markers(segment)
632
633     # generate reads
634     reads = generate_multi_reads(segment, num_reads, ids_rates)
635
636     # MRS pre filter if requested
637     if mode.endswith("+mrs"):
638         aligned_all = run_msa(reads)
639         reads       = mrs_filter(aligned_all, w=4, K=7)
640
641     # final alignment & consensus
642     aligned = run_msa(reads)
643     cons    = majority_vote(aligned)
644
645     # extract payload with fuzzy threshold=0.75
646     if "marker" in mode:
647         cons = extract_payload_with_markers(cons, threshold=0.75)
648         if cons is None:
649             return {"success": False, "reason": "marker lost"}
650
651     # CRT decode
652     decoded = decode_consensus_with_rrns(cons)
653     if decoded is None:
654         return {"success": False, "reason": "CRT failure"}
655
656     # metrics
657     return {
658         "success": True,
659         "BER": bit_error_rate(clean, decoded),
660         "edit": normalized_edit_distance(clean, decoded)
661     }
662
663 # sanity checks again
664 print(run_pipeline(segments[0], mode="rrns+msa"))
665 print(run_pipeline(segments[0], mode="rrns+marker"))
666 print(run_pipeline(segments[0], mode="rrns+msa+mrs"))
667
668 # %%
669 # ==== Cell 26      sweep, aggregate & plot =====
670 import pandas as pd
671 import matplotlib.pyplot as plt
672
673 def sweep(
674     error_rates=(0.02, 0.05, 0.1),
675     modes=("rrns+msa", "rrns+marker", "rrns+msa+mrs"),
676     samples=100
677 ) -> pd.DataFrame:
678     """
679     For each IDS rate and each method, run 'run_pipeline' on the first 'samples' segments
680     and collect only successful BER/edit results into a DataFrame.
681     """
682     rows = []
683     for er in error_rates:
684         rate = {"ins": er, "del": er, "sub": er}
685         for mode in modes:
686             for seg in segments[:samples]:
687                 res = run_pipeline(seg, num_reads=8, mode=mode, ids_rates=rate)
688                 if res.get("success"):
689                     rows.append({
690                         "mode": mode,
691                         "err": er,
692                         "BER": res["BER"],
693                         "edit": res["edit"]
694                     })

```

```

695     return pd.DataFrame(rows)
696
697 # 1) Run the sweep
698 df = sweep(error_rates=(0.02, 0.05), samples=50)
699
700 # 2) Aggregate mean std of BER for each mode & error rate
701 summary = (
702     df
703     .groupby(["mode", "err"])
704     .BER
705     .agg(mean="mean", std="std", count="count")
706     .reset_index()
707 )
708
709 # 3) Plot mean BER 1 error bars
710 fig, ax = plt.subplots(figsize=(6,4))
711 for mode, sub in summary.groupby("mode"):
712     sub = sub.sort_values("err")
713     ax.errorbar(
714         sub["err"],
715         sub["mean"],
716         yerr=sub["std"],
717         marker="o",
718         label=mode,
719         capsize=3
720     )
721
722 ax.set_xlabel("IDS error rate")
723 ax.set_ylabel("Mean BER 1 ")
724 ax.set_title("Comparison of RRNS-based methods")
725 ax.legend()
726 plt.tight_layout()
727 plt.show()
728
729 # %%
730 # ==== Cell 27 full sweep & final plot =====
731 import pandas as pd
732 import matplotlib.pyplot as plt
733
734 # 1) Run the sweep on 100 segments
735 df_full = sweep(
736     error_rates=(0.02, 0.05, 0.10),
737     modes=("rrns+msa", "rrns+marker", "rrns+msa+mrs"),
738     samples=100
739 )
740
741 # 2) Aggregate
742 summary_full = (
743     df_full
744     .groupby(["mode", "err"])
745     .BER.agg(["mean", "std"])
746     .reset_index()
747 )
748
749 # 3) Plot
750 fig, ax = plt.subplots(figsize=(6,4))
751 for mode, sub in summary_full.groupby("mode"):
752     sub = sub.sort_values("err")
753     ax.errorbar(
754         sub["err"], sub["mean"], yerr=sub["std"],
755         marker="o", capsize=3, label=mode
756     )
757 ax.set_xlabel("IDS probability")
758 ax.set_ylabel("Mean BER 1 ")
759 ax.set_title("Final RRNS Performance (BL=12, MOD=[7,11,13,17,19])")
760 ax.legend()
761 plt.tight_layout()
762 plt.show()
763
764 # %%
765 # ==== Cell X parameter sweep for BLOCK_LEN & MODULI =====
766
767 import pandas as pd
768
769 # 1) define the parameter grid you want to test
770 block_lengths = [8, 10, 12]
771 moduli_sets = [
772     [5, 7, 11, 13, 17], # original
773     [3, 5, 7, 11, 13], # alt 5 mod 11 set
774     [7, 11, 13, 17, 19], # ba ka bir varyasyon
775 ]
776 error_rates = [0.02, 0.05]

```

```

778 results = []
779
780 # 2) sweep over GRID
781 for bl in block_lengths:
782     for mods in moduli_sets:
783         # override globals
784         BLOCK_LEN = bl
785         MODULI = mods
786
787         for er in error_rates:
788             rates = {"ins": er, "del": er, "sub": er}
789
790             # run ONLY rrns+msa for now
791             res = run_pipeline(
792                 segments[0],
793                 num_reads=8,
794                 mode="rrns+msa",
795                 ids_rates=rates
796             )
797             if res.get("success"):
798                 results.append({
799                     "BLOCK_LEN": bl,
800                     "MODULI": " ".join(map(str, mods)),
801                     "err": er,
802                     "BER": res["BER"],
803                     "edit": res["edit"],
804                 })
805
806 # 3) collect into DataFrame and display
807 df_params = pd.DataFrame(results)
808 display(df_params.pivot_table(
809     index=["BLOCK_LEN", "MODULI"],
810     columns="err",
811     values="BER",
812     aggfunc="mean"
813 ))
814
815 # %%
816 # ==== Cell X+1      full parameter sweep across methods =====
817 import pandas as pd
818
819 # same grid
820 block_lengths = [8, 10, 12]
821 moduli_sets = [
822     [5, 7, 11, 13, 17],
823     [3, 5, 7, 11, 13],
824     [7, 11, 13, 17, 19],
825 ]
826 error_rates = [0.02, 0.05]
827 modes = ["rrns+msa", "rrns+marker", "rrns+msa+mrs"]
828
829 results = []
830
831 for bl in block_lengths:
832     for mods in moduli_sets:
833         # override the constants
834         BLOCK_LEN = bl
835         MODULI = mods
836
837         for er in error_rates:
838             rates = {"ins": er, "del": er, "sub": er}
839
840             for mode in modes:
841                 res = run_pipeline(
842                     segments[0],
843                     num_reads=8,
844                     mode=mode,
845                     ids_rates=rates
846                 )
847                 # only record successful runs
848                 if res.get("success"):
849                     results.append({
850                         "BLOCK_LEN": bl,
851                         "MODULI": " ".join(map(str, mods)),
852                         "mode": mode,
853                         "err": er,
854                         "BER": res["BER"],
855                         "edit": res["edit"],
856                     })
857
858 # aggregate and pivot
859 df_full = pd.DataFrame(results)
860

```

```
861 pivot = df_full.pivot_table(  
862     index=["BLOCK_LEN", "MODULI", "mode"],  
863     columns="err",  
864     values="BER",  
865     aggfunc="mean"  
866 )  
867 display(pivot)
```

Listing 3. RRNS + MRS Pipeline – Implemented by Emre